CATEGORY B

Part A — Coding Problems

Problem 1: Bank Account Withdrawal System

TASK

Design a system for bank accounts with withdrawal functionality. There are two types of accounts: Savings and Current. A Savings account must maintain a minimum balance of 1000 after any withdrawal. A Current account can have an overdraft up to 5000 (meaning its balance can go down to -5000). Implement a withdraw operation that rejects withdrawals if they violate these rules. Assume initial balances are always positive.

INPUT / OUTPUT

Input Format: The first line contains a series of account operations. Each operation is either:

- Savings <ID> <InitialBalance> to create a Savings account.
- Current <ID> <InitialBalance> to create a Current account.
- WITHDRAW <ID> <Amount> to attempt a withdrawal.

Output Format: For each WITHDRAW operation, print the status of the withdrawal and the current balance if successful, or the reason for rejection. If an account is not found, print 'Account not found'.

EXAMPLE

```
Sample Input: Savings S1 5000
              WITHDRAW S1 4500
              Current C1 2000
              WITHDRAW C1 6000
              WITHDRAW S1 3000
              WITHDRAW C1 2000

Expected Output: S1 rejected: minimum balance 1000
                 C1 balance -4000
                 S1 balance 2000
                 C1 rejected: overdraft limit 5000
```

Problem 2: Campus Vehicle Pass System

TASK

Implement a system to manage campus vehicle passes and charging bay bookings. Every vehicle has a pass number and an owner. The yearly pass fee varies by vehicle type:

- Bike: 300
- Car: 1000
- E-Bike: 300
- E-Car: 1000

Only electric vehicles (E-Bike, E-Car) are eligible to book a charging bay. Design classes for vehicles and their types, incorporating an interface for chargeable vehicles.

INPUT / OUTPUT

Input Format: Each line describes an operation:

• PASS <VehicleType> <PassNumber> <OwnerName>: To register a vehicle and print its pass fee. • CHARGE <PassNumber>: To attempt to book a charging bay for the vehicle with the given pass number.

Output Format: For each PASS operation, print the vehicle's pass number, type, and calculated pass fee. For each CHARGE operation, print whether the charging bay was allotted or if charging is unsupported for that vehicle.

EXAMPLE

```
Sample Input: PASS Bike KA01 Asha
              PASS ECar KA02 Ravi
              CHARGE KA02
              CHARGE KA01

Expected Output: KA01 (Bike) pass fee 300
                 KA02 (ECar) pass fee 1000
                 KA02 charging bay allotted
                 KA01 rejected: charging unsupported
```

Problem 3: Student Club Registration Management

TASK

Develop a student club registration system that stores members by their unique roll number. The system must prevent duplicate registrations for the same student, even if the registration details (roll number and name) arrive as a new object instance. Ensure that when you check if a student is already registered, only the roll number is considered for uniqueness. After processing operations, report the total number of unique members and check if a specific student is present.

INPUT / OUTPUT

Input Format: Each line contains an operation:

• ADD <RollNumber> <Name>: To add a new student member.

• CONTAINS <RollNumber> <Name>: To check if a student with the given roll number is registered.

Output Format: For ADD operations, print 'Added' if the student is new, or 'duplicate rejected' if already present. After all ADD operations, print 'member count <count>' with the total number of unique members. For CONTAINS operations, print 'contains: true' or 'contains: false'.

EXAMPLE

```
Sample Input: ADD 21CS01 Asha
              ADD 21CS01 Asha
              ADD 21CS02 Ravi
              CONTAINS 21CS02 Ravi
              CONTAINS 21CS03 Priya

Expected Output: Added
                 duplicate rejected
                 Added
                 member count 2
                 contains: true
                 contains: false
```

Problem 4: Class Marks Grid Analysis

TASK

Create a program to manage and analyze student marks in multiple subjects, stored in a 2D array (grid). For a given set of student marks, calculate and report each student's total marks, the average marks for each subject, and identify the student with the highest total marks (the topper).

INPUT / OUTPUT

Input Format: Multiple lines, each representing a student's marks: <StudentName> [<Subject1Marks>, <Subject2Marks>, ...] Assume there are always 3 subjects for simplicity.

Output Format:

- Print the total marks for each student.
- Print the average marks for each subject, formatted to two decimal places.
- Print the name of the topper along with their total marks.

EXAMPLE

```
Sample Input: Asha [78, 85, 90]
              Ravi [88, 92, 79]
              Neha [65, 70, 95]

Expected Output: Totals Asha 253, Ravi 259, Neha 230
                 averages 77.00, 82.33, 88.00
                 topper Ravi (259)
```

Problem 5: Payroll Register System

TASK

Develop a payroll system for different staff types: FullTime, PartTime, and Intern. Each staff member has a name. Their pay is calculated as follows:

- FullTime: A fixed monthly salary.
- PartTime: Hours worked multiplied by an hourly rate.
- Intern: A fixed stipend.

Store all staff members' payslips in an array. Each payslip should have a readable text representation showing the name, type, and calculated pay. After processing all staff, print each staff member's payslip and identify the top earner.

INPUT / OUTPUT

Input Format: Each line describes a staff member:

- FullTime <Name> <Salary>
- PartTime <Name> <Hours> <Rate>
- Intern <Name> <Stipend>

Output Format: For each staff member, print their payslip in the format:

Payslip[name=<Name>, type=<Type>, pay=<CalculatedPay>]. Then, print the 'Total <SumOfAllPay>' and 'top earner <NameOfTopper>'.

EXAMPLE

```
Sample Input: FullTime Asha 50000
              PartTime Ravi 80 300
              Intern Neha 15000

Expected Output: Payslip[name=Asha, type=FullTime, pay=50000]
                 Payslip[name=Ravi, type=PartTime, pay=24000]
                 Payslip[name=Neha, type=Intern, pay=15000]
                 Total 89000
                 top earner Asha
```

Part B — Quiz

Choose the best answer for each question.

Q1. Consider the given UML class diagram. If Student is a derived type of Person, which members are directly accessible within the Student class?

```
Class Person {
  - name: String
```

```
  # age: int
  + getName(): String
}
Class Student --|> Person
```

A.- name

B.# age and + getName()

C.- name, # age, and + getName()

D.+ getName() only

Q2. Examine the three UML diagrams representing class hierarchies. Which diagram illustrates an invalid class inheritance structure, assuming a language that supports single class inheritance?

Diagram 1:

```
Class A
Class B --|> A
Class C --|> B
```

Diagram 2:

```
Class X
Class Y --|> X
Class Z --|> X
```

Diagram 3:

```
Class P
Class Q --|> P
Class R --|> P
Class S --|> Q
Class S --|> R
```

A.Diagram 1, representing multilevel inheritance.

B.Diagram 2, representing hierarchical inheritance.

C.Diagram 3, where a class attempts to extend two different classes.

D.All diagrams represent valid inheritance structures.

Q3. Based on the sequence diagram for creating a Manager object, which represents a derived type, in what order will the constructors be executed? Assume Manager extends Employee, and Employee extends Person.

Sequence Diagram:

```
new Manager()
  Manager() calls Employee()
    Employee() calls Person()
```


A.Manager -> Employee -> Person

B.Person -> Employee -> Manager

C.Employee -> Manager -> Person

D.Person -> Manager -> Employee

Q4. Consider a Derived class inheriting from a Base class as shown in the UML. Which of the following statements about accessing Base members or modifying Base operations from within Derived is correct?

```
Class Base {
  - privateField: int
  # protectedField: int
  + publicMethod(): void
  + {final} finalMethod(): void
}
Class Derived --|> Base {
  // Attempting to access members and override finalMethod
}
```

A.Derived can directly access privateField.

B.Derived can override finalMethod().

C.Derived can directly access protectedField.

D.Derived can weaken the access modifier of publicMethod() when overriding it.

Q5. Given the UML class diagram where Bus extends Vehicle, and both classes define a fare() method. If a Vehicle reference points to a Bus object, and fare() is called through this reference, which version of the fare() operation will run?

```
Class Vehicle {
  + fare(): double
}
Class Bus --|> Vehicle {
  + fare(): double
}
```

A.The fare() method defined in Vehicle.

B.The fare() method defined in Bus.

C.A compile-time error will occur.

D.The method called depends on how the reference is declared.

Q6. In the given UML diagram, Printer has two print methods with different parameters, and SpecialPrinter redefines one of them. Which statement correctly identifies the distinction between overloading and overriding?

```
Class Printer {
  + print(text: String): void
  + print(data: int): void
}
Class SpecialPrinter --|> Printer {
  + print(text: String): void // Redefines inherited print(String)
}
```


A.print(text: String) in Printer is overridden, while print(data: int) is overloaded. B.print(text: String) and print(data: int) in Printer are examples of overloading, and SpecialPrinter's print(text: String) is an example of overriding.

C.SpecialPrinter's print(text: String) is an example of overloading.

D.Overloading is chosen at run-time, while overriding is chosen at compile-time.

Q7. If a Shape reference s is assigned a Circle object (upcasting), which of the following members can

be accessed directly using the s reference?

```
Class Shape {
  // ...
}
Class Circle --|> Shape {
  + radius: double
  + calculateArea(): double
}
```

A.Only members defined in the Shape class.

B.Both members defined in Shape and Circle, including radius.

C.Only members defined in Circle.

D.radius specifically, as it's a public attribute.

Q8. Consider an array shapes of type Shape that holds a Circle object at index 0 and a Square object at index 1. Both Circle and Square override the area() method inherited from Shape. What is the expected behavior when a loop iterates through shapes and calls area() on each element?

Object Diagram:

```
Shape[] shapes = new Shape[2];
shapes[0] = new Circle();
shapes[1] = new Square();

Class Shape {
  + area(): double {abstract}
}
Class Circle --|> Shape {
  + area(): double // Returns specific area for Circle
}
Class Square --|> Shape {
  + area(): double // Returns specific area for Square
}
```

A.All calls to area() will invoke the area() method of the Shape class.

B.A compile-time error will occur because Shape has an abstract area() method. C.The area() method specific to Circle will be called for shapes[0], and the area() method specific to Square will be called for shapes[1].

D.The behavior is undefined and depends on the specific runtime environment.

Q9. Given an Account reference a that currently refers to a Current object. Which of the following casting attempts will succeed at runtime, and which will fail?

Object Diagram:

```
Account a = new Current(); // 'a' is a reference to a Current object
Savings s;
Current c;
```


A.s = (Savings) a; will succeed, c = (Current) a; will fail.

B.s = (Savings) a; will fail, c = (Current) a; will succeed.

C.Both s = (Savings) a; and c = (Current) a; will succeed.

D.Both s = (Savings) a; and c = (Current) a; will fail.

Q10. An array plans of type Plan contains objects of three different subtypes: PrepaidPlan,

PostpaidPlan, and FamilyPlan. Each subtype overrides a bill() method to calculate charges differently. What is the benefit of processing these objects in a loop calling bill() on each Plan element?

Sequence Diagram:

```
loop over Plan[] elements
  call bill() on each element
```

A. It requires a series of if-else if statements to determine the actual type of each plan before calling its bill() method.

B. It allows uniform processing of different derived objects through a base-type reference, without needing type-specific conditional logic.

C. It compiles faster than calling the bill() method directly on each specific type.

D. It prevents any modification to the objects within the array.

Q11. Consider a Printer class with the members shown in the UML diagram. Which of the following operations represents the essential behavior exposed to a caller, and which represent hidden details of 'how' the printer works?

```
Class Printer {
  + printDocument(doc: Document): void
  - internalBuffer: byte[]
  - loadBuffer(doc: Document): void
  - sendToPrintHead(): void
}
```

A. printDocument() is essential behavior; internalBuffer, loadBuffer(), and sendToPrintHead() are hidden details.

B. internalBuffer is essential behavior; printDocument() is a hidden detail.

C. All operations and attributes are essential behavior.

D. Only - internalBuffer is a hidden detail; all methods are essential behavior.

Q12. Given the UML diagram, where Vehicle is an abstract class and Car is a concrete subclass. Which statements are true regarding the instantiation and referencing of these types? `{abstract} Class Vehicle {`

```
    // ...
}
Class Car --|> Vehicle {
  // ...
}
```

A. An object of type Vehicle can be directly instantiated.

B. A Vehicle reference can refer to a Car object.

C. A Car reference can refer to a Vehicle object without casting.

D. Car cannot be instantiated because Vehicle is abstract.

Q13. In the given UML, Account is an abstract class with an abstract operation calculateFee(). SavingsAccount is a subclass of Account. If SavingsAccount does not implement calculateFee(), what will be the consequence?

```
{abstract} Class Account {
```

```
  + {abstract} calculateFee(): double
}
Class SavingsAccount --|> Account {
  // Missing implementation of calculateFee()
}
```

A.SavingsAccount will automatically inherit a default implementation of calculateFee().

B.SavingsAccount must also be declared as an abstract class.

C.The program will compile, but a runtime error will occur when calculateFee() is called on a SavingsAccount object.

D.SavingsAccount can be instantiated without any issues.

Q14. Given the abstract class Report and its subclass MonthlyReport as described in the UML. Which of the following scenarios would lead to a compilation error?

```
{abstract} Class Report {
  - reportId: String
  + Report(id: String)
  + generateHeader(): String
  + {abstract} generateBody(): String
  // No implementation for generateBody()
}
Class MonthlyReport --|> Report {
  // Missing implementation for generateBody()
}
```

A.Report having a constructor and a concrete method (generateHeader()).

B.MonthlyReport attempting to be a concrete class without implementing the abstract method generateBody().

C.Declaring Report as abstract even if it contains a concrete method.

D.Creating an instance of MonthlyReport after it correctly implements generateBody().

Q15. Based on the UML diagram for the Payable interface, what does an interface fundamentally specify and what does it typically not provide?

```
<<interface>> Payable {
  + getAmount(): double
  + pay(): void
}
```

A.It specifies operations a type promises to provide, but not per-object state or direct implementation.

B.It specifies per-object state and provides a default implementation for all

operations. C.It specifies how an object is created and its initial state.

D.It only specifies abstract operations and cannot contain constants.

Q16. Given the Loggable interface in the UML diagram, which statement accurately describes the implicit visibility and modifiers of its members?

```
<<interface>> Loggable {
```

```
  MAX_LOG_SIZE: int = 1024
  + logMessage(message: String): void
  + getLogLevel(): int
}
Class Logger implements Loggable {
  // ...
}
```

A.MAX_LOG_SIZE is implicitly private static final.

B.logMessage() and getLogLevel() are implicitly public abstract.

C.All members, including MAX_LOG_SIZE, are implicitly public static.

D.Interface members have no implicit modifiers and must always be explicitly declared.

Q17. A DataProcessor class realizes both the Serializable and Resettable interfaces as shown in the UML diagram. Which operations must the DataProcessor class implement to fulfill its contracts?

```
<<interface>> Serializable {
   + serialize(): byte[]
}
<<interface>> Resettable {
  + reset(): void
}
Class DataProcessor ..|> Serializable
Class DataProcessor ..|> Resettable {
  // ...
}
```

A.Only serialize() from Serializable.

B.Only reset() from Resettable.

C.Both serialize() from Serializable and reset() from Resettable.

D.No operations, as interfaces only define contracts, not mandatory implementations.

Q18. Consider a requirement for a type that needs to define a common protocol for unrelated classes to adhere to, without providing any default implementation or holding per-object state. Which type of construct would be the most appropriate choice for this requirement?

```
Box A: Class MyClass { ... }
Box B: {abstract} Class MyAbstractClass { ... }
Box C: <<interface>> MyInterface { ... }
Box D: (Unlabeled, representing a requirement)
```

A.Box A (Concrete Class)

B.Box B (Abstract Class)

C.Box C (Interface)

D.Any of the above could fulfill the requirement with minor adjustments.

Q19. In the given UML diagram, classify the relationships between Dog and Animal, and Eagle and Flyable.

```
Class Animal
Class Dog --|> Animal // Generalization
Class Bird
<<interface>> Flyable
Class Eagle ..|> Flyable // Realization
Class Dog --> Leash // Association
```

A.Dog IS-A Animal (generalization); Eagle IS-A Flyable (generalization).

B.Dog IS-A Animal (generalization); Eagle CAN-DO Flyable (realization).

C.Dog CAN-DO Animal (realization); Eagle CAN-DO Flyable (realization).

D.Dog CAN-DO Animal (association); Eagle IS-A Flyable (association).

Q20. In many object-oriented languages, all classes implicitly inherit from a root type (e.g., Object in Java). Which of the following operations are commonly inherited by default by every object from this root type, and are often redefined for specific class behavior?

```
Class MyObject {
  // ... no explicit extends clause
}
```

A.equals(), hashCode(), toString()

B.Only hashCode() and toString()

C.Only equals()

D.clone(), finalize(), wait()

Part C — Concept Questions

Answer each question in a few sentences, giving reasons for your answer.

Question 1

Explain the fundamental difference between method overloading and method overriding in object-oriented programming, particularly focusing on when the specific method implementation is resolved (compile-time vs. run-time).

Question 2

Describe the sequence of constructor invocation when a derived class object is created. If a derived constructor needs to pass values to its base constructor, how is this typically achieved?

Question 3

Compare and contrast concrete classes, abstract types, and interfaces based on their ability to be instantiated, hold state, and provide implemented behavior. Under what circumstances would you choose each for designing a type in a software system?

Question 4

A class implements two different interfaces, both of which declare an abstract method with the identical signature. Explain how the implementing class must handle these methods and what this demonstrates about interface implementation.

Question 5

When defining custom objects, explain why it is crucial to override both the equals() and hashCode() methods if you intend to define 'content equality' rather than 'identity equality'. What are the potential consequences if equals() is overridden but hashCode() is not, especially when using hash-based collections?

Question 6

Differentiate between shallow copying and deep copying of an object. Illustrate a scenario where a shallow copy might lead to unexpected behavior if the original object contains mutable nested objects, and explain why a deep copy would be necessary in such a case.

Question 7

Explain the key distinctions between an inner type and a static nested type regarding their instantiation and their ability to access members of the enclosing class. Provide a suitable use case for each type of nested class.

Question 8

Discuss the time complexity of the following array operations: element access by index, and insertion/deletion of an element in the middle of a fixed-size array. Explain the underlying reasons for their respective Big-O notations.

Question 9

Describe how an array containing objects of different derived types can be processed uniformly using a base-type reference. How does dynamic method dispatch enable different behaviors for the same method call across these varied objects without explicit type checks?

Question 10

When designing a system, how do you decide whether to model a relationship between types using an 'IS-A' relationship (inheritance) or a 'CAN-DO' relationship (interfaces)? Provide an example scenario where an interface would be a more appropriate choice than inheritance.